Git Cheat Sheet — Version Control for Reproducible Research

Quick Command Reference

What you want to do	Command
Start a repo	`git init`
Check status	`git status`
Stage files	`git add .`
Commit	`git commit -m "msg"`
See history	`git log --oneline`
Tag a milestone	`git tag -a v1.0 -m "note"`
Switch branches	`git checkout branch-name`
Create a branch	`git checkout -b name`
Merge branch "name" into current branch	`git merge name`
Push to Code Hosting Service	`git push origin main`
Pull from Code Hosting Service	`git pull origin main`
Undo uncommitted edits	`git checkout -- file`

There are a few code hosting services that integrate with Git. The most popular is GitHub, but there are also GitLab, Bitbucket, Codeberg, and others. All the Git commands below that mention Github will work with any of these services.

First-Time Setup (do this once)

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

[optional] git config --global init.defaultBranch main
```

> **Note:** Use the email associated with your GitHub account for proper attribution, but you can use any email address if needed.

Starting a Project

Task	Command
Create a new repo in the current folder	`git init`
Clone an existing repo from GitHub	`git clone <url>`
Check the status of your repo	`git status`

The Core Workflow: Edit → Stage → Commit

Think of it like photography: you **arrange** what goes in the shot (stage), then **take the picture** (commit).

Task	Command
Stage a specific file	`git add filename.R`
Stage everything that changed	`git add .`
Commit staged changes with a message	`git commit -m "Describe what changed"`
Stage + commit in one step (tracked files only)	`git commit -am "Describe what changed"`

Writing Good Commit Messages

Do this ☑	Not this ❌
`"Add linear model for Figure 2"`	`"stuff"`
`"Fix sample size filter (n>30, was n>20)"`	`"updated code"`
`"Clean raw data: remove NAs from weight col"`	`"changes"`

> **Tip:** Write messages that your future self will thank you for in 6 months.

Viewing History

Task	Command
See full commit history	`git log`
Compact one-line history	`git log --oneline`
See history for the n most recent commits	`git log -n 5`
See **which files** changed in each commit	`git log --name-only`
See **what changed** in each commit	`git log --stat`
See the diff of uncommitted changes	`git diff`
See who changed what line (useful for collabs!)	`git blame <filename>`

Tags: Marking Milestones

Tags are bookmarks for important moments — manuscript submissions, analysis freezes, releases.

Task	Command
Tag the current commit	`git tag v1.0-submitted`
Tag with an annotation/message	`git tag -a v1.0-submitted -m "Submitted to Nature"`
List all tags	`git tag`
Push tags to GitHub	`git push origin --tags`

Example naming conventions:

- `v1.0-submitted` — first manuscript submission
- `v1.1-revised` — post-review revision
- `analysis-freeze-2026-03` — data analysis locked

Branches

Branches let you try things without breaking what works.

Task	Command
See all branches	`git branch`
Create a new branch	`git branch new-analysis`
Switch to a branch	`git checkout new-analysis`
Create + switch in one step	`git checkout -b new-analysis`
Merge a branch back into main	`git checkout main` then `git merge new-analysis`
Delete a branch after merging	`git branch -d new-analysis`

Collaborating with GitHub

Task	Command
Link your local repo to GitHub	`git remote add origin <url>`
Push your commits to GitHub	`git push origin main`
Pull collaborator's changes	`git pull origin main`
See what remotes are configured	`git remote -v`

Typical collaboration flow:

```
1.  git pull origin main          ← get latest changes
2.  (do your work, edit files)
3.  git add .                     ← stage
4.  git commit -m "message"       ← commit
5.  git push origin main          ← share your work
```

> **Golden rule:** Always `pull` before you `push`.

Oops! Fixing Mistakes

Situation	Command
Undo changes to a file (not yet staged)	`git checkout -- filename`
Unstage a file (keep the edits)	`git reset HEAD filename`
Amend your last commit message	`git commit --amend -m "New message"`
Revert a commit (safe — creates a new commit)	`git revert <commit-hash>`
See a file from an old commit	`git show <commit-hash>:filename`

> ⚠ **Avoid `git reset --hard` unless you know what you're doing** — it permanently deletes uncommitted work.

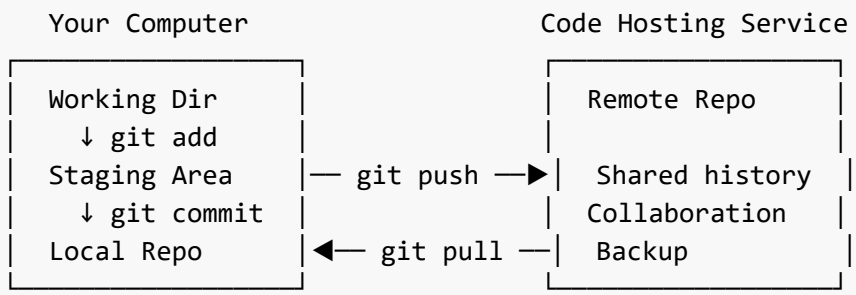
The `.gitignore` File

Create a file called `.gitignore` in your repo to tell Git what **not** to track:

```
# Large data files
data/raw/*.csv

# System files
.DS_Store
Thumbs.db

# Sensitive info
credentials.yaml

# R/Python outputs
*.RData
__pycache__/
.ipynb_checkpoints/
```

> **Tip:** Never commit passwords, API keys, large data files, or generated outputs.

🗺 Mental Model



BOOST — Version Control for Reproducible Research — March 11, 2026